

```
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.payment.entity.PaymentBatch;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface PaymentBatchMapper extends BaseMapper<PaymentBatch> {
}

package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.employee.entity.Employee;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface EmployeeMapper extends BaseMapper<Employee> {
}

package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.rbac.entity.SysUserRole;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface SysUserRoleMapper extends BaseMapper<SysUserRole> {
}

package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.system.entity.IntegrationConfig;
public interface IntegrationConfigMapper extends BaseMapper<IntegrationConfig> {
}

package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.payroll.entity.PayrollImportItem;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface PayrollImportItemMapper extends BaseMapper<PayrollImportItem> {}

package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.rbac.entity.SysRole;
```

```

import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface SysRoleMapper extends BaseMapper<SysRole> {}
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.rbac.entity.SysResource;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Param;
import org.apache.ibatis.annotations.Update;
import java.util.List;
@Mapper
public interface SysResourceMapper extends BaseMapper<SysResource> {
    @Update("<script>" +
        "UPDATE sys_resource SET order_num = CASE id " +
        "<foreach collection='items' item='item' separator=' '>" +
        "WHEN #{item.id} THEN #{item.orderNum} " +
        "</foreach>" +
        "END WHERE id IN " +
        "<foreach collection='items' item='item' open='(' separator=',' close=')'>#{item.id}</foreach>" +
        "</script>")
    int batchUpdateOrderNum(@Param("items") List<SortItemDto> items);
    record SortItemDto(Long id, Integer orderNum) {}
}
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.payment.entity.PaymentRecord;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface PaymentRecordMapper extends BaseMapper<PaymentRecord> {
}
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.org.entity.Department;
import org.apache.ibatis.annotations.Mapper;
@Mapper

```

```

public interface DepartmentMapper extends BaseMapper<Department> {
}
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.payroll.entity.SalaryTemplate;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface SalaryTemplateMapper extends BaseMapper<SalaryTemplate> {}
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.payroll.entity.TimesheetEntry;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface TimesheetEntryMapper extends BaseMapper<TimesheetEntry> {}
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.audit.dto.AuditLogQueryRequest;
import com.yiyundao.compensation.modules.audit.entity.AuditLog;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Param;
import org.apache.ibatis.annotations.Select;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
@Mapper
public interface AuditLogMapper extends BaseMapper<AuditLog> {
    @Select("SELECT * FROM audit_log WHERE username = #{username} ORDER BY create_time DESC LIMIT #{limit}")
    List<AuditLog> findRecentByUsername(@Param("username") String username, @Param("limit") int limit);
    @Select("SELECT COUNT(*) FROM audit_log WHERE username = #{username} " +
        "AND operation LIKE CONCAT('%', 'ç\231»â¼\225', '%') " +
        "AND response_result = 'FAILED' " +
        "AND create_time >= #{startTime}")
    int countLoginFailures(@Param("username") String username, @Param("startTime") LocalDateTime startTime);
    @Select("SELECT response_result, COUNT(*) as count FROM audit_log " +
        "WHERE operation LIKE CONCAT('%', 'ç\231»â¼\225', '%') " +

```

```

        "AND DATE(create_time) = CURDATE() " +
        "GROUP BY response_result")
List<Map<String, Object>> getTodayLoginStats();
@Select("SELECT operation, COUNT(*) as count FROM audit_log " +
        "WHERE create_time >= #{startTime} AND create_time <= #{endTime} " +
        "GROUP BY operation ORDER BY count DESC")
List<Map<String, Object>> getOperationStats(@Param("startTime") LocalDateTime startTime,
        @Param("endTime") LocalDateTime endTime);
List<AuditLog> findByTimeRange(@Param("startTime") LocalDateTime startTime,
        @Param("endTime") LocalDateTime endTime);
List<AuditLog> queryByPage(@Param("request") AuditLogQueryRequest request);
}

package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.payroll.entity.PayrollAdjustment;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface PayrollAdjustmentMapper extends BaseMapper<PayrollAdjustment> {}

package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.rbac.entity.SysRoleResource;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface SysRoleResourceMapper extends BaseMapper<SysRoleResource> {
}

package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.interfaces.vo.payroll.PayrollBatchSummaryVO;
import com.yiyundao.compensation.modules.payroll.entity.PayrollBatch;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Param;
import java.util.List;
@Mapper
public interface PayrollBatchMapper extends BaseMapper<PayrollBatch> {
    List<PayrollBatchSummaryVO> selectBatchSummaryList(

```

```

        @Param("type") String type,
        @Param("periodLabel") String periodLabel,
        @Param("status") String status,
        @Param("offset") int offset,
        @Param("size") int size
    );
    long countBatchSummary(@Param("type") String type, @Param("periodLabel") String periodLabel, @Param("status") String status);
}

package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.system.entity.SysConfig;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface SysConfigMapper extends BaseMapper<SysConfig> {
}

package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.approval.entity.ApprovalWorkflow;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface ApprovalWorkflowMapper extends BaseMapper<ApprovalWorkflow> {
}

package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.rbac.entity.SysUserResource;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface SysUserResourceMapper extends BaseMapper<SysUserResource> {
}

package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.user.entity.SysUser;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface SysUserMapper extends BaseMapper<SysUser> {
}

```

```

}
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.payroll.entity.PayrollLine;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface PayrollLineMapper extends BaseMapper<PayrollLine> {}
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.notification.entity.NotificationRecord;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface NotificationRecordMapper extends BaseMapper<NotificationRecord> {
}package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.employee.entity.EmployeeDepartment;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface EmployeeDepartmentMapper extends BaseMapper<EmployeeDepartment> {}
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.approval.entity.ApprovalStep;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface ApprovalStepMapper extends BaseMapper<ApprovalStep> {
}
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.app.entity.AppRegistry;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface AppRegistryMapper extends BaseMapper<AppRegistry> {
}
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;

```

```

import com.yiyundao.compensation.modules.payroll.entity.PayCycle;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface PayCycleMapper extends BaseMapper<PayCycle> {}
package com.yiyundao.compensation.infrastructure.dao;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.yiyundao.compensation.modules.payroll.entity.SalaryItem;
import org.apache.ibatis.annotations.Mapper;
@Mapper
public interface SalaryItemMapper extends BaseMapper<SalaryItem> {}
package com.yiyundao.compensation.scheduler;
import com.yiyundao.compensation.entity.ScheduledTask;
import com.yiyundao.compensation.entity.ScheduledTaskExecution;
import com.yiyundao.compensation.mapper.ScheduledTaskExecutionMapper;
import com.yiyundao.compensation.service.ScheduledTaskService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.scheduling.concurrent.ThreadPoolTaskScheduler;
import org.springframework.stereotype.Component;
import jakarta.annotation.PostConstruct;
import java.time.LocalDateTime;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ScheduledFuture;
@Slf4j
@Component
@RequiredArgsConstructor
public class TaskScheduler {
    private final ScheduledTaskService taskService;
    private final ThreadPoolTaskScheduler taskScheduler;
    private final ScheduledTaskExecutionMapper executionMapper;
    private final Map<Long, ScheduledFuture<?>> scheduledTasks = new ConcurrentHashMap<>();
    @PostConstruct
    public void init() {
        try {

```

```

        taskService.listTasksByStatus(ScheduledTask.TaskStatus.RUNNING)
            .forEach(this::scheduleTask);
        log.info("è°\203Å°!ä»»Å\212;Å\210\235Å$ \213Å\214\226Å@ \214æ \210\220İ¼ \214Å•²Å\212 è¼½ {} ä,ªä»»Å\212; ", scheduledTasks
.size());
    } catch (Exception e) {
        log.warn("è°\203Å°!ä»»Å\212;Å\210\235Å$ \213Å\214\226Å±è´¥İ¼ \210Å\217`è\203¼æ \230`æ \225°æ \215@Å° \223è; ¨æ \234ª \210\233
Å»°İ¼ \211İ¼ \214Å° \206Å \234¨è; ¨Å\210\233Å»°Å \220\216è \207ª \212¨Å\212 è¼½: {}", e.getMessage());
    }
}

public void scheduleTask(ScheduledTask task) {
    try {
        ScheduledFuture<?> future = taskScheduler.schedule(
            () -> executeTask(task),
            triggerContext -> {
                LocalDateTime nextTime = taskService.calculateNextExecutionTime(task.getCronExpression());
                if (nextTime != null) {
                    return nextTime.atZone(java.time.ZoneId.systemDefault()).toInstant();
                }
                return java.time.LocalDateTime.now().plusMinutes(5).atZone(java.time.ZoneId.systemDefault()).toInstant();
            }
        );
        scheduledTasks.put(task.getId(), future);
        log.info("ä»»Å\212;Å•²è°\203Å°!: taskKey={}", task.getTaskKey());
    } catch (Exception e) {
        log.error("è°\203Å°!ä»»Å\212;Å±è´¥: taskKey={}", task.getTaskKey(), e);
    }
}

private void executeTask(ScheduledTask task) {
    log.info("Å¼\200Å$ \213æ \211$è; \214ä»»Å\212;: taskKey={}", task.getTaskKey());
    long startTime = System.currentTimeMillis();
    ScheduledTaskExecution execution = new ScheduledTaskExecution();
    execution.setTaskId(task.getId());
    execution.setTaskKey(task.getTaskKey());
    execution.setStartTime(LocalDateTime.now());
    execution.setStatus(ScheduledTaskExecution.ExecutionStatus.RUNNING);
}

```



```

executionMapper.insert(execution);
try {
    taskService.executeTask(task);
    execution.setEndTime(LocalDateTime.now());
    execution.setDurationMs(System.currentTimeMillis() - startTime);
    execution.setStatus(ScheduledTaskExecution.ExecutionStatus.SUCCESS);
    execution.setResult("æ\211$è;\214æ\210\220â\212\237");
    executionMapper.updateById(execution);
    log.info("â»â\212;æ\211$è;\214æ\210\220â\212\237: taskKey={}", task.getTaskKey());
} catch (Exception e) {
    log.error("â»â\212;æ\211$è;\214â±è´¥: taskKey={}", task.getTaskKey(), e);
    execution.setEndTime(LocalDateTime.now());
    execution.setDurationMs(System.currentTimeMillis() - startTime);
    execution.setStatus(ScheduledTaskExecution.ExecutionStatus.FAILED);
    execution.setErrorMessage(e.getMessage());
    executionMapper.updateById(execution);
    if (task.getAlarmEnabled()) {
        sendAlarm(task, e.getMessage());
    }
}
}

private void sendAlarm(ScheduledTask task, String error) {
    log.warn("â»â\212;æ\211$è;\214â±è´¥â\221\212è-!: taskKey={}, error={}", task.getTaskKey(), error);
}

public void pauseTask(Long taskId) {
    ScheduledFuture<?> future = scheduledTasks.get(taskId);
    if (future != null) {
        future.cancel(false);
        scheduledTasks.remove(taskId);
        log.info("â»â\212;â•²æ\232\202â\201\234: taskId={}", taskId);
    }
}

public void resumeTask(ScheduledTask task) {
    scheduleTask(task);
}

```

```

}
package com.yiyundao.compensation.enums;
import lombok.Getter;
@Getter
public enum PlatformType {
    WECHAT("wechat", "ã¼\201ä,\232ã¼@ä;"),
    DINGTALK("dingtalk", "é\222\211é\222\211"),
    FEISHU("feishu", "éf\236ä¹");
    private final String code;
    private final String name;
    PlatformType(String code, String name) {
        this.code = code;
        this.name = name;
    }
    public static PlatformType fromCode(String code) {
        for (PlatformType type : values()) {
            if (type.getCode().equals(code)) {
                return type;
            }
        }
        throw new IllegalArgumentException("Unknown platform type: " + code);
    }
}
package com.yiyundao.compensation.enums;
import lombok.Getter;
@Getter
public enum ApprovalStatus {
    PENDING("pending", "ã¼\205ã¼@;æ\211:"),
    APPROVED("approved", "ã•²é\200\232è¿\207"),
    REJECTED("rejected", "ã•²æ\213\222ç»\235"),
    CANCELLED("cancelled", "ã•²ã\217\226æ¶\210"),
    SKIPPED("skipped", "ã•²è•³è¿\207");
    private final String code;
    private final String name;
    ApprovalStatus(String code, String name) {
        this.code = code;
    }
}

```

```

        this.name = name;
    }
    public static ApprovalStatus fromCode(String code) {
        for (ApprovalStatus status : values()) {
            if (status.getCode().equals(code)) {
                return status;
            }
        }
        throw new IllegalArgumentException("Unknown approval status: " + code);
    }
}
package com.yiyundao.compensation.enums;
import com.baomidou.mybatisplus.annotation.EnumValue;
import lombok.Getter;
@Getter
public enum NotificationStatus {
    PENDING("pending", "â%205â217221é200201"),
    SENDING("sending", "â217221é200201â.-"),
    SUCCESS("success", "â217221é200201â210220â212237"),
    FAILED("failed", "â217221é200201â±è´¥"),
    RETRY("retry", "é207215è225â.-"),
    CANCELLED("cancelled", "â•²â217226æ¶210");
    @EnumValue
    private final String code;
    private final String name;
    NotificationStatus(String code, String name) {
        this.code = code;
        this.name = name;
    }
}
public static NotificationStatus fromCode(String code) {
    for (NotificationStatus status : values()) {
        if (status.getCode().equals(code)) {
            return status;
        }
    }
    throw new IllegalArgumentException("Unknown notification status: " + code);
}

```

```

    }
}package com.yiyundao.compensation.enums;
import com.baomidou.mybatisplus.annotation.EnumValue;
import lombok.Getter;
@Getter
public enum PayrollType {
    FULL_TIME("full_time", "à\205"è\201\214è\226*èµ\204"),
    PART_TIME("part_time", "à\205%è\201\214è\226*èµ\204"),
    BONUS("bonus", "å¥\226é\207\221"),
    REIMBURSEMENT("reimbursement", "æ\212¥é\224\200");
    @EnumValue
    private final String code;
    private final String name;
    PayrollType(String code, String name) {
        this.code = code;
        this.name = name;
    }
    public static PayrollType fromCode(String code) {
        if (code == null) return null;
        for (PayrollType type : values()) {
            if (type.getCode().equalsIgnoreCase(code)) {
                return type;
            }
        }
        return null;
    }
}
package com.yiyundao.compensation.enums;
import com.baomidou.mybatisplus.annotation.EnumValue;
import lombok.Getter;
@Getter
public enum PayrollBatchStatus {
    DRAFT("draft", "è\215\211ç":"),
    LOCKED("locked", "å•²é\224\201â@\232"),
    SUBMITTED("submitted", "å•²æ\217\220ä°ªâ@;æ\211:");
}

```

```

APPROVED("approved", "å•²å@;æ\211¹"),
REJECTED("rejected", "å•²æ\213\222ç»\235"),
PAY_PROCESSING("pay_processing", "æ\224²ä»\230å¼\204ç\220\206ä,-"),
PAID("paid", "å•²æ\224²ä»\230"),
ARCHIVED("archived", "å•²å¼\222æ;f");
@EnumValue
private final String code;
private final String name;
PayrollBatchStatus(String code, String name) {
    this.code = code;
    this.name = name;
}
public static PayrollBatchStatus fromCode(String code) {
    if (code == null) return null;
    for (PayrollBatchStatus status : values()) {
        if (status.getCode().equalsIgnoreCase(code)) {
            return status;
        }
    }
    return null;
}
public boolean canEdit() {
    return this == DRAFT;
}
public boolean canLock() {
    return this == DRAFT;
}
public boolean canSubmitApproval() {
    return this == LOCKED;
}
public boolean canCompute() {
    return this == LOCKED || this == APPROVED;
}
}
package com.yiyundao.compensation.enums;

```

```

import com.baomidou.mybatisplus.annotation.EnumValue;
import lombok.Getter;
@Getter
public enum NotificationType {
    PAYMENT_SUCCESS("payment_success", "æ\224`ä»\230æ\210\220â\212\237é\200\232ç\237¥"),
    PAYMENT_FAILED("payment_failed", "æ\224`ä»\230â±è´¥é\200\232ç\237¥"),
    BATCH_COMPLETE("batch_complete", "æ\211¹æ¬;â@\214æ\210\220é\200\232ç\237¥"),
    APPROVAL_PENDING("approval_pending", "â¼\205â@;æ\211¹é\200\232ç\237¥"),
    APPROVAL_APPROVED("approval_approved", "â@;æ\211¹é\200\232è¿\207é\200\232ç\237¥"),
    APPROVAL_REJECTED("approval_rejected", "â@;æ\211¹æ\213\222ç»\235é\200\232ç\237¥"),
    SYSTEM_ALERT("system_alert", "ç³»ç»\237â\221\212è-;é\200\232ç\237¥");
    @EnumValue
    private final String code;
    private final String name;
    NotificationType(String code, String name) {
        this.code = code;
        this.name = name;
    }
    public static NotificationType fromCode(String code) {
        for (NotificationType type : values()) {
            if (type.getCode().equals(code)) {
                return type;
            }
        }
        throw new IllegalArgumentException("Unknown notification type: " + code);
    }
}
package com.yiyundao.compensation.enums;
import com.baomidou.mybatisplus.annotation.EnumValue;
import com.fasterxml.jackson.annotation.JsonValue;
import lombok.Getter;
@Getter
public enum UserStatus {
    ACTIVE("active", "æ¿\200æ»"),
    INACTIVE("inactive", "ç¿\201ç\224");
    @EnumValue

```

```

@JsonValue
private final String code;
private final String name;
UserStatus(String code, String name) {
    this.code = code;
    this.name = name;
}
}
public static UserStatus fromCode(String code) {
    for (UserStatus status : values()) {
        if (status.getCode().equals(code)) {
            return status;
        }
    }
    throw new IllegalArgumentException("Unknown user status: " + code);
}
}

package com.yiyundao.compensation.enums;
import com.baomidou.mybatisplus.annotation.EnumValue;
import lombok.Getter;
@Getter
public enum PaymentType {
    SALARY("salary", "â•¥è\204"),
    BONUS("bonus", "â¥\226é\207\221"),
    REIMBURSEMENT("reimbursement", "æ\212¥é\224\200");
    @EnumValue
    private final String code;
    private final String name;
    PaymentType(String code, String name) {
        this.code = code;
        this.name = name;
    }
}
public static PaymentType fromCode(String code) {
    for (PaymentType type : values()) {
        if (type.getCode().equals(code)) {
            return type;
        }
    }
}

```

```

    }
    }
    throw new IllegalArgumentException("Unknown payment type: " + code);
}
}package com.yiyundao.compensation.enums;
import com.baomidou.mybatisplus.annotation.EnumValue;
import lombok.Getter;
@Getter
public enum PaymentStatus {
    PENDING("pending", "â%205â%204ç220206"),
    PROCESSING("processing", "â%204ç220206ä.-"),
    SUCCESS("success", "æ210220â212237"),
    FAILED("failed", "â±è"),
    CANCELLED("cancelled", "â•â217226æ210");
    @EnumValue
    private final String code;
    private final String name;
    PaymentStatus(String code, String name) {
        this.code = code;
        this.name = name;
    }
    public static PaymentStatus fromCode(String code) {
        for (PaymentStatus status : values()) {
            if (status.getCode().equals(code)) {
                return status;
            }
        }
        throw new IllegalArgumentException("Unknown payment status: " + code);
    }
}package com.yiyundao.compensation.enums;
import lombok.Getter;
@Getter
public enum EmployeeStatus {
    ACTIVE("active", "â234è201214"),
    INACTIVE("inactive", "ç|è201214"),

```



```

    SUSPENDED("suspended", "å\201\234è\201\214");
    private final String code;
    private final String name;
    EmployeeStatus(String code, String name) {
        this.code = code;
        this.name = name;
    }
    public static EmployeeStatus fromCode(String code) {
        for (EmployeeStatus status : values()) {
            if (status.getCode().equals(code)) {
                return status;
            }
        }
        throw new IllegalArgumentException("Unknown employee status: " + code);
    }
}
package com.yiyundao.compensation.enums;
import com.baomidou.mybatisplus.annotation.EnumValue;
import lombok.Getter;
@Getter
public enum EmploymentType {
    FULL_TIME("full_time", "å\205"è\201\214"),
    PART_TIME("part_time", "å\205¼è\201\214"),
    INTERN("intern", "å@\236å: "),
    CONTRACT("contract", "å\220\210å\220\214å•¥");
    @EnumValue
    private final String code;
    private final String name;
    EmploymentType(String code, String name) {
        this.code = code;
        this.name = name;
    }
    public static EmploymentType fromCode(String code) {
        if (code == null) return null;
        for (EmploymentType type : values()) {
            if (type.getCode().equalsIgnoreCase(code)) {

```

```

        return type;
    }
}
return null;
}
}

package com.yiyundao.compensation.enums;
import lombok.Getter;
@Getter
public enum WorkflowType {
    BATCH("BATCH", "æ\211'é\207\217æ\224`ä»\230"),
    ADHOC("ADHOC", "ä.´æ\227¶æ\224`ä»\230"),
    OFFLINE("OFFLINE", "ç!»ç°¿â\221\230ä•¥");
    private final String code;
    private final String name;
    WorkflowType(String code, String name) {
        this.code = code;
        this.name = name;
    }
    public static WorkflowType fromCode(String code) {
        for (WorkflowType type : values()) {
            if (type.getCode().equals(code)) {
                return type;
            }
        }
        throw new IllegalArgumentException("Unknown workflow type: " + code);
    }
}

package com.yiyundao.compensation.enums;
import com.baomidou.mybatisplus.annotation.EnumValue;
import lombok.Getter;
@Getter
public enum NotificationChannel {
    WECHAT("wechat", "ä¼\201ä.\232ä%ä¿;"),
    DINGTALK("dingtalk", "é\222\211é\222\211"),
    FEISHU("feishu", "é£\236ä'");
}

```

```

SMS("sms", "ç\237-ä;"),
EMAIL("email", "é\202@ä»"),
SYSTEM("system", "ç«\231â\206\205ä;");
@EnumValue
private final String code;
private final String name;
NotificationChannel(String code, String name) {
    this.code = code;
    this.name = name;
}
}
public static NotificationChannel fromCode(String code) {
    for (NotificationChannel channel : values()) {
        if (channel.getCode().equals(code)) {
            return channel;
        }
    }
    throw new IllegalArgumentException("Unknown notification channel: " + code);
}
}
package com.yiyundao.compensation.enums;
import com.baomidou.mybatisplus.annotation.EnumValue;
import lombok.Getter;
@Getter
public enum BatchStatus {
    DRAFT("draft", "è\215\211ç";"),
    SUBMITTED("submitted", "â•²æ\217\220ä°¤"),
    APPROVED("approved", "â•²â@;æ\211¹"),
    PROCESSING("processing", "â¤\204ç\220\206ä,-"),
    COMPLETED("completed", "â•²â@\214æ\210\220"),
    FAILED("failed", "â¤±è´¥");
    @EnumValue
    private final String code;
    private final String name;
    BatchStatus(String code, String name) {
        this.code = code;
        this.name = name;
    }
}

```

```

    }
    public static BatchStatus fromCode(String code) {
        for (BatchStatus status : values()) {
            if (status.getCode().equals(code)) {
                return status;
            }
        }
        throw new IllegalArgumentException("Unknown batch status: " + code);
    }
}

package com.yiyundao.compensation.security;
import com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.yiyundao.compensation.common.response.ApiResponse;
import com.yiyundao.compensation.infrastructure.dao.SysResourceMapper;
import com.yiyundao.compensation.infrastructure.dao.SysUserMapper;
import com.yiyundao.compensation.infrastructure.dao.SysUserRoleMapper;
import com.yiyundao.compensation.modules.rbac.entity.SysResource;
import com.yiyundao.compensation.modules.rbac.entity.SysUserRole;
import com.yiyundao.compensation.modules.rbac.service.ResourceService;
import com.yiyundao.compensation.modules.rbac.service.UserRoleService;
import com.yiyundao.compensation.modules.user.entity.SysUser;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.util.AntPathMatcher;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

```

```

@Slf4j
@RequiredArgsConstructor
public class ApiResourceAuthorizationFilter extends OncePerRequestFilter {
    private final SysResourceMapper sysResourceMapper;
    private final SysUserMapper sysUserMapper;
    private final SysUserRoleMapper userRoleMapper;
    private final ResourceService resourceService;
    private final UserRoleService userRoleService;
    private final ObjectMapper objectMapper;
    private final AntPathMatcher pathMatcher = new AntPathMatcher();
    private volatile List<SysResource> apiResourcesCache;
    private final AtomicLong lastLoadTs = new AtomicLong(0);
    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
        throws ServletException, IOException {
        String method = request.getMethod();
        String uri = request.getRequestURI(); // e.g. /api/payment/batch/123/start
        String servletPath = request.getServletPath(); // e.g. /payment/batch/123/start
        log.debug("ApiResourceAuthorizationFilter: method={}, uri={}, servletPath={}", method, uri, servletPath);
        SysResource matched = matchApiResource(method, uri, servletPath);
        log.debug("ApiResourceAuthorizationFilter: matched={}", matched != null ? matched.getCode() : "null");
        if (matched == null) {
            log.debug("ApiResourceAuthorizationFilter: No API resource matched, passing through");
            filterChain.doFilter(request, response);
            return;
        }
        var auth = org.springframework.security.core.context.SecurityContextHolder.getContext().getAuthentication();
        log.debug("ApiResourceAuthorizationFilter: auth={}, isAuthenticated={}", auth, auth != null ? auth.isAuthenticated() : "null");
        if (auth == null || !auth.isAuthenticated()) {
            writeJson(response, 401, "æ\234ªç\231»â%\225");
            return;
        }
        String username = auth.getName();
        SysUser user = sysUserMapper.selectOne(new LambdaQueryWrapper<SysUser>().eq(SysUser::getUsername, username).last("limit 1"

```

```

));
    if (user == null) {
        writeJson(response, 401, "æ\234ªç\231»â¼\225");
        return;
    }
    if (isAdminUser(user.getId(), user)) {
        filterChain.doFilter(request, response);
        return;
    }
    boolean hasResource = resourceService.getUserResources(user.getId())
        .stream().anyMatch(r -> r.getId().equals(matched.getId()));
    if (!hasResource) {
        writeJson(response, 403, "æ\227 æ\235\203é\231\220è@ç\227@è`¥æ\216¥â\217£");
        return;
    }
    List<String> actions = resourceService.getUserActions(user.getId()).get(matched.getId());
    if (actions == null || actions.isEmpty()) {
        filterChain.doFilter(request, response);
        return;
    }
    String requiredAction = mapHttpMethodToAction(method);
    boolean hasPermission = actions.contains("*") || actions.contains(requiredAction) || actions.contains(method.toUpperCase());
};
    if (!hasPermission) {
        writeJson(response, 403, "æ\227 æ\235\203é\231\220æ\211$è; \214è`¥æ\223\215ä¼\234i¼\210é\234\200è; \201 " + requiredAction + " æ\235\203é\231\220i¼\211");
        return;
    }
    filterChain.doFilter(request, response);
}
private String mapHttpMethodToAction(String httpMethod) {
    return switch (httpMethod.toUpperCase()) {
        case "GET" -> "read";
        case "POST" -> "write";
        case "PUT" -> "write";
    };
}

```

```

        case "DELETE" -> "delete";
        case "PATCH" -> "write";
        default -> httpMethod.toLowerCase();
    };
}
private boolean isAdminUser(Long userId, SysUser user) {
    return userRoleService.hasRole(userId, SecurityConstants.ROLE_ADMIN);
}
private SysResource matchApiResource(String method, String uri, String servletPath) {
    long now = System.currentTimeMillis();
    if (apiResourcesCache == null || (now - lastLoadTs.get()) > SecurityConstants.API_RESOURCE_CACHE_TTL_MS) {
        apiResourcesCache = sysResourceMapper.selectList(new LambdaQueryWrapper<SysResource>()
            .eq(SysResource::getType, "API")
            .eq(SysResource::getStatus, "enabled"));
        lastLoadTs.set(now);
    }
    for (SysResource r : apiResourcesCache) {
        if (!StringUtils.hasText(r.getPath())) continue;
        String pattern = r.getPath();
        String requiredMethod = extractMethod(r.getPropsJson());
        if (requiredMethod != null && !requiredMethod.equalsIgnoreCase(method)) continue;
        if (pathMatcher.match(pattern, uri)) {
            return r;
        }
    }
    if (!pattern.startsWith("/api")) {
        if (pathMatcher.match(pattern, servletPath)) {
            return r;
        }
        String normalized = pattern.startsWith("/") ? "/api" + pattern : "/api/" + pattern;
        if (pathMatcher.match(normalized, uri)) {
            return r;
        }
    }
    } else {
        String withoutPrefix = pattern.substring(4);
        if (!withoutPrefix.startsWith("/")) {

```

```

        withoutPrefix = "/" + withoutPrefix;
    }
    if (pathMatcher.match(withoutPrefix, servletPath)) {
        return r;
    }
}
return null;
}
private String extractMethod(String propsJson) {
    if (!StringUtils.hasText(propsJson)) return null;
    try {
        JsonNode node = objectMapper.readTree(propsJson);
        JsonNode m = node.get("method");
        return m == null ? null : m.asText();
    } catch (Exception ignored) {
        return null;
    }
}
private void writeJson(HttpServletResponse response, int code, String message) throws IOException {
    response.setStatus(200); // ä.\216â\205"â±\200â\223\215â°\224æ¼â¼\217â¿\235æ\214\201â,\200è\207´
    response.setCharacterEncoding(StandardCharsets.UTF_8.name());
    response.setContentType("application/json; charset=UTF-8");
    String body = objectMapper.writeValueAsString(ApiResponse.error(code, message));
    response.getWriter().write(body);
}
}
package com.yiyundao.compensation.security;
import com.yiyundao.compensation.common.config.ExternalApiAuthProperties;
import com.yiyundao.compensation.modules.app.entity.AppRegistry;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import jakarta.annotation.PostConstruct;
import lombok.RequiredArgsConstructor;

```



```

import org.springframework.security.core.authority.AuthorityUtils;
import org.springframework.stereotype.Service;
import org.springframework.util.Assert;
import org.springframework.util.CollectionUtils;
import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.util.ArrayList;
import java.util.Date;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.UUID;
@Service
@RequiredArgsConstructor
public class ExternalApiTokenService {
    private static final String ROLE_APP = "ROLE_APP";
    private final ExternalApiAuthProperties properties;
    private SecretKey secretKey;
    private long expirationSeconds;
    @PostConstruct
    void init() {
        String secret = properties.getJwt().getSecret();
        Assert.hasText(secret, "external-api.jwt.secret must be configured");
        this.secretKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.expirationSeconds = Math.max(properties.getJwt().getExpirationSeconds(), 60L);
    }
    public TokenResult generateToken(AppRegistry app, List<String> requestedScopes) {
        Assert.notNull(app, "app must not be null");
        List<String> scopes = normalizeScopes(requestedScopes);
        if (CollectionUtils.isEmpty(scopes)) {
            throw new IllegalArgumentException("æ\234ªæ\216\210æ\235\203ë@çé\227@è\214\203â\233´");
        }
        Instant now = Instant.now();
        Instant expiry = now.plusSeconds(expirationSeconds);
    }
}

```

```

String authorities = buildAuthorityString(scopes);
String token = Jwts.builder()
    .id(UUID.randomUUID().toString())
    .subject(app.getClientId())
    .issuedAt(Date.from(now))
    .expiration(Date.from(expiry))
    .claim("appId", app.getId())
    .claim("appName", app.getAppName())
    .claim("scp", scopes)
    .claim("authorities", authorities)
    .signWith(secretKey)
    .compact();
return new TokenResult(token, expiry, scopes);
}

public ParsedToken parseToken(String token) {
    Claims claims = Jwts.parser()
        .verifyWith(secretKey)
        .build()
        .parseSignedClaims(token)
        .getPayload();
    String clientId = claims.getSubject();
    Long appId = convertLong(claims.get("appId"));
    String appName = claims.get("appName", String.class);
    List<?> scopeRaw = claims.get("scp", List.class);
    List<String> scopes = new ArrayList<>();
    if (scopeRaw != null) {
        scopeRaw.forEach(item -> {
            if (item != null) {
                scopes.add(item.toString());
            }
        });
    }
    String authorities = claims.get("authorities", String.class);
    Instant expiresAt = claims.getExpiration() != null ? claims.getExpiration().toInstant() : null;
    return new ParsedToken(clientId, appId, appName, scopes, authorities, expiresAt);
}

```

```

}
private List<String> normalizeScopes(List<String> scopes) {
    if (CollectionUtils.isEmpty(scopes)) {
        return List.of();
    }
    Set<String> normalized = new LinkedHashSet<>();
    for (String scope : scopes) {
        if (scope != null) {
            String trimmed = scope.trim();
            if (!trimmed.isEmpty()) {
                normalized.add(trimmed);
            }
        }
    }
    return new ArrayList<>(normalized);
}
private String buildAuthorityString(List<String> scopes) {
    List<String> authorities = new ArrayList<>();
    authorities.add(ROLE_APP);
    for (String scope : scopes) {
        authorities.add("SCOPE_" + scope);
    }
    return String.join(",", authorities);
}
private Long convertLong(Object value) {
    if (value == null) {
        return null;
    }
    if (value instanceof Number number) {
        return number.longValue();
    }
    return Long.parseLong(value.toString());
}
public record TokenResult(String accessToken, Instant expiresAt, List<String> scopes) {
    public long expiresInSeconds() {

```

```

        return expiresAt != null ? Math.max(0, expiresAt.getEpochSecond() - Instant.now().getEpochSecond()) : 0;
    }
    public String scopeString() {
        return String.join(" ", scopes);
    }
}
public record ParsedToken(String clientId,
                          Long appId,
                          String appName,
                          List<String> scopes,
                          String authorities,
                          Instant expiresAt) {
    public boolean isExpired() {
        return expiresAt != null && expiresAt.isBefore(Instant.now());
    }
    public List<org.springframework.security.core.authority.SimpleGrantedAuthority> toAuthorities() {
        if (authorities == null || authorities.isEmpty()) {
            return List.of();
        }
        return AuthorityUtils.commaSeparatedStringToAuthorityList(authorities)
            .stream()
            .map(auth -> new org.springframework.security.core.authority.SimpleGrantedAuthority(auth.getAuthority()))
            .toList();
    }
}
}
package com.yiyundao.compensation.security;
import lombok.Builder;
import lombok.Data;
import org.springframework.stereotype.Component;
@Component
public class ExternalApiContext {
    private static final ThreadLocal<ExternalApiClient> CONTEXT = new ThreadLocal<>();
    public void set(ExternalApiClient client) {
        CONTEXT.set(client);
    }
}

```

```
}
public ExternalApiClient current() {
    return CONTEXT.get();
}
public void clear() {
    CONTEXT.remove();
}
@Data
@Builder
public static class ExternalApiClient {
    private Long appId;
    private String clientId;
    private String appName;
    private String clientIp;
    private java.util.List<String> scopes;
}
}
package com.yiyundao.compensation.security;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.yiyundao.compensation.common.response.ApiResponse;
import com.yiyundao.compensation.common.response.ErrorCode;
import com.yiyundao.compensation.modules.app.entity.AppRegistry;
import com.yiyundao.compensation.modules.app.service.AppRateLimitService;
import com.yiyundao.compensation.modules.app.service.AppRegistryService;
import com.yiyundao.compensation.modules.app.service.impl.AppRateLimitServiceImpl;
import com.yiyundao.compensation.modules.audit.service.AuditLogService;
```

```

    private String department;
    private Long managerId;
    private String keyword;
    private Integer totalEmployees;
    private BigDecimal earningsTotal;
    private BigDecimal deductionsTotal;
    private BigDecimal grossTotal;
    private BigDecimal taxTotal;
    private BigDecimal socialTotal;
    private BigDecimal netTotal;
    private Integer linesWithWarnings;
    private Integer totalWarnings;
    private List<String> warnings;
    private List<PayrollPreviewDto.PayrollPreviewLineDto> lines;
}

package com.yiyundao.compensation.interfaces.dto.payroll;
import jakarta.validation.constraints.NotBlank;
import lombok.Data;
import java.time.LocalDate;
@Data
public class PayCycleUpsertRequest {
    @NotBlank
    private String type; // monthly/custom
    @NotBlank
    private String periodLabel; // YYYY-MM
    private LocalDate startDate;
    private LocalDate endDate;
    private LocalDate cutoffDate;
    private String status; // open/closed/archived
}

package com.yiyundao.compensation.interfaces.dto.payroll;
import lombok.Data;
@Data
public class PayrollBatchUpdateRequest {
    private Long payCycleId;

```

```

        private String periodLabel;
        private String scopeJson;
        private String currency;
        private String remark;
    }
}
package com.yiyundao.compensation.interfaces.dto.payroll;
import lombok.Data;
import java.math.BigDecimal;
import java.util.List;
@Data
public class PayrollPreviewDto {
    private Long batchId;
    private String currency;
    private Integer totalEmployees;
    private Integer linesWithWarnings;
    private Integer totalWarnings;
    private List<String> warnings;
    private BigDecimal earningsTotal;
    private BigDecimal deductionsTotal;
    private BigDecimal grossTotal;
    private BigDecimal taxTotal;
    private BigDecimal socialTotal;
    private BigDecimal netTotal;
    private List<PayrollPreviewLineDto> lines;
    @Data
    public static class PayrollPreviewLineDto {
        private Long employeeId;
        private String employeeNo;
        private String employeeName;
        private List<PayrollPreviewItemDto> items;
        private BigDecimal earningsTotal;
        private BigDecimal deductionsTotal;
        private BigDecimal grossAmount;
        private BigDecimal taxAmount;
        private BigDecimal socialAmount;
    }
}

```

```

        private BigDecimal netAmount;
        private List<String> warnings; // ç¼ºå±é¹/é30\210å\200¼/å¼\202å..æ\217\220ç¼º
        private List<String> missingItems; // æ²;æ\235çè\201æ±\202å¼\206ç¼ºå±ç\232\204é¹ç¼\226ç \201
        private DiffSummaryDto diff; // ä.\216ä.\212æ\234\237å¹æ\224
        private String department;
        private Long managerId;
        private String managerName;
        private String employmentType;
    }
    @Data
    public static class PayrollPreviewItemDto {
        private String code;
        private String name;
        private String type; // earning/deduction
        private Boolean taxable;
        private BigDecimal amount;
    }
    @Data
    public static class DiffSummaryDto {
        private BigDecimal lastGrossAmount;
        private BigDecimal lastNetAmount;
        private BigDecimal netDeltaAmount;
        private BigDecimal netDeltaPercent; // -1.00 ~ +1.00
    }
}
package com.yiyundao.compensation.interfaces.dto.payroll;
import jakarta.validation.constraints.NotBlank;
import lombok.Data;
@Data
public class SalaryTemplateUpsertRequest {
    @NotBlank
    private String name;
    @NotBlank
    private String type; // full_time / part_time
    private String itemsJson;
}

```



```

        private String taxRuleJson;
        private String status; // enabled/disabled
    }
}
package com.yiyundao.compensation.interfaces.dto.payroll;
import lombok.Data;
import java.math.BigDecimal;
import java.util.List;
@Data
public class PayrollLedgerDto {
    private Long batchId;
    private String status;
    private String periodLabel;
    private String currency;
    private Integer totalEmployees;
    private BigDecimal earningsTotal;
    private BigDecimal deductionsTotal;
    private BigDecimal grossTotal;
    private BigDecimal taxTotal;
    private BigDecimal socialTotal;
    private BigDecimal netTotal;
    private Integer linesWithWarnings;
    private Integer totalWarnings;
    private List<String> warnings;
    private List<PayrollPreviewDto.PayrollPreviewLineDto> lines;
}
package com.yiyundao.compensation.interfaces.dto.payroll;
import lombok.Data;
import java.math.BigDecimal;
import java.util.List;
@Data
public class PayrollBasicReportDto {
    private Long batchId;
    private String periodLabel;
    private String currency;
    private BigDecimal earningsTotal;

```

```

private BigDecimal deductionsTotal;
private BigDecimal grossTotal;
private BigDecimal taxTotal;
private BigDecimal socialTotal;
private BigDecimal netTotal;
private Integer employeeCount;
private List<DepartmentSummary> departments;
@Data
public static class DepartmentSummary {
    private String department;
    private Integer employeeCount;
    private BigDecimal earningsTotal;
    private BigDecimal deductionsTotal;
    private BigDecimal grossTotal;
    private BigDecimal taxTotal;
    private BigDecimal socialTotal;
    private BigDecimal netTotal;
}
}
package com.yiyundao.compensation.interfaces.dto.payroll;
import jakarta.validation.constraints.NotBlank;
import lombok.Data;
@Data
public class PayrollBatchCreateRequest {
    private Long payCycleId;
    private String periodLabel; // 217-206\227ä¼31ä¼ 205¥
    @NotBlank
    private String type; // full_time / part_time
    private String scopeJson; // JSON string
    private String currency; // 230 CNY
    private String remark;
}
package com.yiyundao.compensation.interfaces.dto.org;
import lombok.Data;
import java.util.List;

```

```

@Data
public class OrgDeptTreeResponse {
    private String platformType;
    private List<OrgDeptNodeDto> roots;
}
package com.yiyundao.compensation.interfaces.dto.org;
import lombok.Data;
@Data
public class EmployeePreviewDto {
    private String platformType;
    private String platformUserId;
    private String employeeId;
    private String name;
    private String phone;
    private String email;
    private String department;
    private java.util.List<String> departments;
    private String position;
    private String employmentType; // full_time/part_time
    private String groupKey;
}
package com.yiyundao.compensation.interfaces.dto.org;
import lombok.Data;
import java.util.List;
@Data
public class OrgFetchPreviewResponse {
    private String platformType;
    private int totalEmployees;
    private List<EmployeePreviewDto> employees;
}
package com.yiyundao.compensation.interfaces.dto.org;
import lombok.Data;
import java.util.List;
@Data
public class OrgImportRequest {

```

```

private String platformType;
private List<OrgImportRequest.EmployeeItem> items;
@Data
public static class EmployeeItem {
    private String employeeId;
    private String name;
    private String phone;
    private String email;
    private java.util.List<String> departments;
    private String position;
    private String employmentType; // full_time/part_time
    private String platformUserId;
    private String platformType;
    private String status; // optional
    private Boolean offline; // optional
    private Long managerId; // optional
    private String bankAccount; // optional
    private String bankName; // optional
    private java.time.LocalDate hireDate; // optional
    private String username; // preferred username for system account
}
}
package com.yiyundao.compensation.interfaces.dto.org;
import lombok.Data;
@Data
public class OrgMemberPreviewDto {
    private String platformUserId;
    private String name;
    private String employeeId; // é€\204è$\210é\230fæ@µé\200\232Å. .ä. °ç°°i¼\214ä¼\233ç¼\226è¼\221
    private String phone;
    private String email;
    private String position;
    private String employmentType; // full_time/part_timei¼\214Å\217~ç¼\226è¼\221
}
package com.yiyundao.compensation.interfaces.dto.org;

```

```
import lombok.Data;
import java.util.ArrayList;
import java.util.List;
@Data
public class OrgDeptNodeDto {
    private String platformDeptId;
    private String name;
    private String parentPlatformDeptId;
    private List<OrgDeptNodeDto> children = new ArrayList<>();
    private List<OrgMemberPreviewDto> members = new ArrayList<>();
}
package com.yiyundao.compensation.interfaces.dto.org;
import lombok.Data;
@Data
public class OrgFetchRequest {
    private String platform; // wechat|dingtalk|feishu
}
package com.yiyundao.compensation.interfaces.dto.admin;
import lombok.Data;
import java.time.LocalDateTime;
@Data
public class RoleListItemDto {
    private Long id;
    private String code;
    private String name;
    private String description;
    private String status;
    private Integer userCount;
    private Integer resourceCount;
    private LocalDateTime createTime;
    private LocalDateTime updateTime;
}
package com.yiyundao.compensation.interfaces.dto.admin;
import lombok.Data;
import java.util.List;
```

```

@Data
public class ResourceDetailDto {
    private Long id;
    private String type;          // MENU/VIEW/ACTION/API
    private String code;
    private String name;
    private String path;
    private String component;
    private String icon;
    private Long parentId;
    private Integer orderNum;
    private String status;        // enabled/disabled
    private List<String> actions; // æ\216\210æ\235\203â\212"ä½\234i¼\210â\217"ä,°ç°i¼\211
}

package com.yiyundao.compensation.interfaces.dto.admin;
import lombok.Data;
@Data
public class UserAggregateDto {
    private Long userId;
    private String username;
    private String realName;
    private String email;
    private String phone;
    private String roles; // comma separated
    private Long employeeId; // employee.id
    private String employeeNo; // employee.employee_id
    private String employeeName; // employee.name
    private String platformType;
    private String platformUserId;
}

package com.yiyundao.compensation.interfaces.dto.auth;
import lombok.Data;
@Data
public class OAuthAuthorizeResponse {
    private String url;

```

```
        private String state;
    }
package com.yiyundao.compensation.interfaces.dto.auth;
import lombok.Data;
import java.util.List;
@Data
public class DevTokenRequest {
    private String username;
    private List<String> roles; // e.g. ["ADMIN","MANAGER"]
    private List<String> authorities; // e.g. ["approval:start","approval:read"]
}
package com.yiyundao.compensation.interfaces.dto.auth;
import lombok.Data;
@Data
public class LoginRequest {
    private String username;
    private String password;
}
package com.yiyundao.compensation.interfaces.dto.auth;
import lombok.Data;
@Data
public class LogoutRequest {
    private String refreshToken; // å\217~é\200\211
}
package com.yiyundao.compensation.interfaces.dto.auth;
import lombok.Data;
@Data
public class RefreshRequest {
    private String refreshToken;
}
package com.yiyundao.compensation.interfaces.dto.auth;
import lombok.Data;
import java.util.List;
@Data
public class LoginResponse {
```

```
        private String token;
        private String refreshToken;
        private String username;
        private List<String> roles;
    }
    package com.yiyundao.compensation.interfaces.dto.approval;
    import jakarta.validation.constraints.NotNull;
    import lombok.Data;
    @Data
    public class CancelWorkflowRequest {
        @NotNull
        private Long operatorId;
        private String reason;
    }
    package com.yiyundao.compensation.interfaces.dto.approval;
    import jakarta.validation.constraints.NotNull;
    import lombok.Data;
    @Data
    public class ApprovalDecisionRequest {
        @NotNull
        private Long approverId;
        private String comment;
    }
    package com.yiyundao.compensation.interfaces.dto.approval;
    import jakarta.validation.constraints.NotBlank;
    import jakarta.validation.constraints.NotNull;
    import lombok.Data;
    import java.util.Map;
    @Data
    public class StartWorkflowRequest {
        @NotBlank
        private String workflowType; // WorkflowType code
        @NotBlank
        private String businessKey;
        @NotBlank
```



```

        private String businessType;
        @NotNull
        private Long initiatorId;
        private Map<String, Object> workflowData;
    }

    package com.yiyundao.compensation.interfaces.dto.role;
    import jakarta.validation.constraints.Size;
    import lombok.Data;
    @Data
    public class RoleBaseRequest {
        @Size(min = 2, max = 100, message = "è$222è\211²\220\215ç$°é\225°\\205é;»\234²2-100¹\213é\227´")
        private String name;
        @Size(max = 500, message = "è$222è\211²\217\217è°é\225°\\215è\203è\205è\207500")
        private String description;
        @Size(max = 100, message = "\233 \207 \207è²\206é\225°\\215è\203è\205è\207100")
        private String icon;
        @Size(max = 500, message = "²\207³²é\225°\\215è\203è\205è\207500")
        private String remarks;
    }

    package com.yiyundao.compensation.interfaces.dto.role;
    import jakarta.validation.constraints.NotEmpty;
    import jakarta.validation.constraints.Size;
    import lombok.Data;
    import java.util.List;
    @Data
    public class RoleResourceAssignRequest {
        @Size(max = 200, message = "\215\225²\\210\206é\205\215ç\232\204èµ\204°\220\225°é\207\217\215è\203è\205è\207200")
        private List<ResourceAssignment> resources;
        private Boolean replaceExisting = true;
        @Data
        public static class ResourceAssignment {
            @NotEmpty(message = "èµ\204°\220ID\215è\203.°ç°°")
            private Long resourceId;
            @Size(max = 20, message = "\223\215\234\235\203é\231\220\225°é\207\217\215è\203è\205è\20720")
            private List<String> actions;
        }
    }

```

```

    }
}
package com.yiyundao.compensation.interfaces.dto.role;
import jakarta.validation.constraints.Pattern;
import lombok.Data;
import lombok.EqualsAndHashCode;
@Data
@EqualsAndHashCode(callSuper = true)
public class UpdateRoleRequest extends RoleBaseRequest {
    @Pattern(regexp = "^(enabled|disabled)$", message = "ç\212\226\200\201â\217ªè\203\226\230` enabled æ\210\226 disabled")
    private String status;
    private Integer sortOrder;
    private Boolean isEditable;
}
package com.yiyundao.compensation.interfaces.dto.role;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import lombok.Data;
import java.util.List;
@Data
public class CreateRoleRequest {
    @NotBlank(message = "è$ \222è\211²ç¼\226ç \201ä,\215è\203½ä, °ç@°")
    @Size(min = 2, max = 50, message = "è$ \222è\211²ç¼\226ç \201é\225¿â°|â¿\205é;»â\234`2-50ä¹\213é\227´")
    @Pattern(regexp = "^[A-Za-z][A-Za-z0-9_]*$", message = "è$ \222è\211²ç¼\226ç \201â¿\205é;»â¥â-\227æ` \215â¼\200â¼´¹¼\214â\217ªè\203½â\214\205â\220«â-\227æ` \215â\200\201æ\225°â-\227â\222\214ä,\213â\210\222ç°¿")
    private String code;
    @NotBlank(message = "è$ \222è\211²â\220\215ç$°ä,\215è\203½ä, °ç@°")
    @Size(min = 2, max = 100, message = "è$ \222è\211²â\220\215ç$°é\225¿â°|â¿\205é;»â\234`2-100ä¹\213é\227´")
    private String name;
    @Size(max = 500, message = "è$ \222è\211²æ\217\217è¿°é\225¿â°|ä,\215è\203½è\205è¿\207500")
    private String description;
    @Pattern(regexp = "^(BUSINESS|CUSTOM)$", message = "è$ \222è\211²ç±»â\236\213â\217ªè\203½æ\230` BUSINESS æ\210\226 CUSTOM")
    private String roleType = "BUSINESS";
    private Integer sortOrder = 0;
}

```

```

        @Size(max = 100, message = "â\233%æ \207æ \207è~\206é\225çâ°|ä, \215è\203%è¶\205èç\207100")
        private String icon;
        @Size(max = 500, message = "â¤\207æ³´é\225çâ°|ä, \215è\203%è¶\205èç\207500")
        private String remarks;
        private List<Long> resourceIds;
        private List<String> actions;
    }

    package com.yiyundao.compensation.interfaces.dto.employee;
    import jakarta.validation.Valid;
    import jakarta.validation.constraints.NotEmpty;
    import lombok.Data;
    import java.util.List;
    @Data
    public class BatchImportRequest {
        @Valid
        @NotEmpty
        private List<EmployeeCreateRequest> employees;
    }

    package com.yiyundao.compensation.interfaces.dto.employee;
    import jakarta.validation.constraints.NotBlank;
    import lombok.Data;
    @Data
    public class BindPlatformRequest {
        @NotBlank
        private String platformUserId;
        @NotBlank
        private String platformType; // wechat/dingtalk/feishu
    }

    package com.yiyundao.compensation.interfaces.dto.employee;
    import lombok.Data;
    import java.time.LocalDate;
    @Data
    public class EmployeeUpdateRequest {
        private String name;
        private String phone;
    }

```

```

        private String email;
        private String idCard; // æ\230\216æ\226\207i¼\214â\220\216ç«-â\212 â-\206
        private String department;
        private String position;
        private LocalDate hireDate;
        private String bankAccount; // æ\230\216æ\226\207i¼\214â\220\216ç«-â\212 â-\206
        private String bankName;
    }
    package com.yiyundao.compensation.interfaces.dto.employee;
    import jakarta.validation.constraints.NotBlank;
    import lombok.Data;
    import java.time.LocalDate;
    @Data
    public class EmployeeCreateRequest {
        @NotBlank
        private String employeeId;
        @NotBlank
        private String name;
        private String phone;
        private String email;
        private String idCard;
        private String department;
        private String position;
        private String employmentType;
        private String platformUserId;
        private String platformType;
        private Long managerId;
        private LocalDate hireDate;
        private String status; // é»\230è@ active
        private String bankAccount; // æ\230\216æ\226\207i¼\214â\220\216ç«-â\212 â-\206
        private String bankName;
        private Boolean offline;
        private String username;
    }
    package com.yiyundao.compensation.interfaces.dto.employee;

```

```
import jakarta.validation.constraints.NotBlank;
import lombok.Data;
@Data
public class UpdateStatusRequest {
    @NotBlank
    private String status; // active/inactive/terminated ç-\211
}

package com.yiyundao.compensation.interfaces.dto.openapi;
import lombok.Builder;
import lombok.Data;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.List;
@Data
@Builder
public class OpenApiPayrollLineDto {
    private Long id;
    private Long batchId;
    private String employeeRef;
    private String employmentType;
    private BigDecimal grossAmount;
    private BigDecimal taxAmount;
    private BigDecimal socialAmount;
    private BigDecimal netAmount;
    private String currency;
    private List<String> departments;
    private String employeeNameMasked;
    private String phoneMasked;
    private LocalDateTime generatedAt;
}

package com.yiyundao.compensation.interfaces.dto.openapi;
import lombok.Builder;
import lombok.Data;
import java.math.BigDecimal;
import java.time.LocalDateTime;
```

```
import java.util.List;
@Data
@Builder
public class OpenApiPayslipDto {
    private Long id;
    private String employeeRef;
    private String period;
    private String employmentType;
    private BigDecimal grossAmount;
    private BigDecimal taxAmount;
    private BigDecimal socialAmount;
    private BigDecimal netAmount;
    private String currency;
    private List<String> departments;
    private String employeeNameMasked;
    private String phoneMasked;
    private LocalDateTime generatedAt;
    private List<PayslipItemDto> items;
    @Data
    @Builder
    public static class PayslipItemDto {
        private String code;
        private String name;
        private String type;
        private Boolean taxable;
        private BigDecimal amount;
        private Boolean showOnPayslip;
        private Integer order;
    }
}

package com.yiyundao.compensation.interfaces.dto.openapi;
import lombok.Builder;
import lombok.Data;
import java.time.LocalDateTime;
@Data
```

```
@Builder
public class OpenApiPayrollBatchDto {
    private Long id;
    private String periodLabel;
    private String type;
    private String status;
    private String currency;
    private Long lineCount;
    private LocalDateTime paidAt;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}

package com.yiyundao.compensation.interfaces.dto.app;
import lombok.Builder;
import lombok.Data;
@Data
@Builder
public class ExternalAppTokenResponse {
    private String accessToken;
    private String tokenType;
    private long expiresIn;
    private String scope;
}

package com.yiyundao.compensation.interfaces.dto.app;
import lombok.Builder;
import lombok.Data;
import java.time.LocalDateTime;
import java.util.List;
@Data
@Builder
public class AppRegistryResponse {
    private Long id;
    private String appName;
    private String clientId;
    private List<String> scopes;
```

```
        private List<String> ipWhitelist;
        private String webhookUrl;
        private String status;
        private LocalDateTime createTime;
        private LocalDateTime updateTime;
    }

package com.yiyundao.compensation.interfaces.dto.app;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import lombok.Data;
import java.util.List;
@Data
public class AppRegistryCreateRequest {
    @NotBlank
    private String appName;
    @NotEmpty
    private List<String> scopes;
    private List<String> ipWhitelist;
    private String webhookUrl;
    private String status;
}

package com.yiyundao.compensation.interfaces.dto.app;
import lombok.Builder;
import lombok.Data;
import java.time.LocalDateTime;
import java.util.List;
@Data
@Builder
public class AppRegistrySecretResponse {
    private Long id;
    private String appName;
    private String clientId;
    private String clientSecret;
    private List<String> scopes;
    private List<String> ipWhitelist;
}
```



```

        private String webhookUrl;
        private String status;
        private LocalDateTime createTime;
        private LocalDateTime updateTime;
    }
    package com.yiyundao.compensation.interfaces.dto.app;
    import lombok.Data;
    import java.util.List;
    @Data
    public class AppRegistryUpdateRequest {
        private String appName;
        private List<String> scopes;
        private List<String> ipWhitelist;
        private String webhookUrl;
        private String status;
    }
    package com.yiyundao.compensation.interfaces.dto.config;
    import lombok.Data;
    @Data
    public class SmsConfigDto {
        private String provider; // æ\234\215å\212;å\225\206: aliyun/tencent/huawei/mock
        private String accessKeyId;
        private String accessKeySecret;
        private String signName; // ç\237-ä¿;ç~¼å\220\215
        private String templateCode; // é»\230è@ª";æ\235¿
        private String secretId;
        private String secretKey;
        private String appId;
        private String sdkAppId;
        private String appKey;
        private String appSecret;
        private String sender;
        private String templateId;
        private String endpoint; // æ\234\215å\212;ç«~ç\202¹
        private String region; // å\234ªå\237\237
    }

```

```

        private Integer dailyLimit; // æ¬217æ¬227¥å¬217\221é\200\201é\231\220å¬210¶
        private Integer rateLimitPerMinute; // æ¬217å¬210\206é\222\237é\231\220æµ\201
        private Boolean enabled; // æ¬230¬å¬220!å¬220¬ç¬224"
    }package com.yiyundao.compensation.interfaces.dto.config;
    import lombok.Data;
    @Data
    public class EmailConfigDto {
        private String host; // SMTPæ¬234\215å¬212;å¬231"
        private Integer port; // ç¬224"æ¬210•å¬220\215
        private String username; // ç¬224"æ¬210•å¬220\215
        private String password; // å¬206ç¬201
        private String fromAddress; // å¬217\221å»¶å¬234°å¬235\200
        private String fromName; // å¬217\221å»¶å¬220\215ç§¬
        private Boolean ssl; // æ¬230¬å¬220!å¬220¬ç¬224"SSL
        private Boolean tls; // æ¬230¬å¬220!å¬220¬ç¬224"TLS
        private String encoding; // ç¬226ç¬201
        private Boolean enabled; // æ¬230¬å¬220!å¬220¬ç¬224"
    }package com.yiyundao.compensation.interfaces.dto.config;
    import lombok.Data;
    import java.time.LocalDateTime;
    @Data
    public class IntegrationConfigListDto {
        private String platformType; // å¬217°ç¬236\213: wechat, dingtalk, feishu, alipay, sms, email, encryption
        private String platformName; // å¬217°æ¬230%ç¬220\215ç§¬
        private Boolean enabled; // æ¬230¬å¬220!å¬220¬ç¬224"
        private Boolean configured; // æ¬230¬å¬220!å¬205\215ç¬20
        private String connectionStatus; // è¬236æ¬216¥ç¬212¶æ¬200\201: connected, disconnected, unknown
        private LocalDateTime lastModified; // æ¬234\200å¬220\216ä¬æ¬224!æ¬227¶é¬227´
    }package com.yiyundao.compensation.interfaces.dto.config;
    import lombok.Data;
    @Data
    public class FeishuConfigDto {
        private String appId;
        private String appSecret;
        private String appToken; // å¬224ç¬224"Token

```

```

        private Boolean enabled;
    }
    package com.yiyundao.compensation.interfaces.dto.config;
    import lombok.Data;
    @Data
    public class WechatConfigDto {
        private String corpId;
        private String corpSecret;
        private String agentId;
    }
    package com.yiyundao.compensation.interfaces.dto.config;
    import lombok.Data;
    @Data
    public class EncryptionConfigDto {
        private String aesKey; // AES\206\222
        private String sm4Key; // SM4\206\222
        private String algorithm; // \227\225: SM4+AES/AES256
        private String keyDerivation; // \206\222\224\237\226\217
        private Integer keyRotationDays; // \206\222\215\225
        private Boolean enabled; // \230\220\220\224
    }
    package com.yiyundao.compensation.interfaces.dto.config;
    import lombok.Data;
    @Data
    public class AlipayConfigDto {
        private String appId;
        private String serverUrl;
        private String privateKey;
        private String publicKey;
        private String charset;
        private String signType;
        private String format;
        private String notifyUrl;
        private String returnUrl;
    }
    package com.yiyundao.compensation.interfaces.dto.config;

```

```

import lombok.Data;
import java.time.LocalDateTime;
@Data
public class IntegrationConfigDetailDto {
    private String platformType; // 17°±36\213
    private String platformName; // 17°230%ç°220\215ç$°
    private Boolean enabled; // 230°220;220°ç\224°
    private Object config; // 205•ä½\223é\205\215ç¼0â\206\205â0°'Ï¼\210æ °æ\2150â°'â\217°ç±»â\236\213è¿\224â\233\236â°'â°\224ç\232\204DT0i¼\211
    private String connectionStatus; // è¿\236æ\216¥ç\212¶æ\200\201: connected, disconnected, unknown
    private LocalDateTime lastModified; // æ\234\200â\220\216ä¿æ\224°æ\227¶é\227´
}package com.yiyundao.compensation.interfaces.dto.config;
import lombok.Data;
@Data
public class DingTalkConfigDto {
    private String appKey;
    private String appSecret;
    private String webhookUrl; // æ\234°â\231°ä°°Webhookâ\234°â\235\200
    private Boolean enabled;
}
package com.yiyundao.compensation.interceptor;
import com.yiyundao.compensation.common.annotation.Idempotent;
import com.yiyundao.compensation.common.response.ApiResponse;
import com.yiyundao.compensation.common.response.ErrorCode;
import com.yiyundao.compensation.service.IdempotentService;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.autoconfigure.condition.ConditionalOnBean;
import org.springframework.stereotype.Component;
import org.springframework.web.method.HandlerMethod;
import org.springframework.web.servlet.HandlerInterceptor;
import java.lang.reflect.Method;
import java.util.HashMap;

```

```

import java.util.Map;
@Slf4j
@Component
@ConditionalOnBean(IdempotentService.class)
public class IdempotentInterceptor implements HandlerInterceptor {
    private final IdempotentService idempotentService;
    @Autowired
    public IdempotentInterceptor(@Autowired(required = false) IdempotentService idempotentService) {
        this.idempotentService = idempotentService;
    }
    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) {
        if (!(handler instanceof HandlerMethod handlerMethod)) {
            return true;
        }
        Method method = handlerMethod.getMethod();
        Idempotent idempotent = method.getAnnotation(Idempotent.class);
        if (idempotent == null) {
            return true;
        }
        if (idempotentService == null) {
            return true;
        }
        String idempotentKey = generateKey(request, idempotent, handlerMethod);
        boolean acquired = idempotentService.tryLock(
            idempotentKey,
            idempotent.expireSeconds(),
            idempotent.tryLock() ? idempotent.waitTime() : 0
        );
        if (!acquired) {
            log.warn("â\202ç-\211æ\200$æf\200æ\237¥æ\234*é\200\232èç\207: key={}", idempotentKey);
            writeResponse(response, ErrorCode.REQUEST_CONFLICT, idempotent.message());
            return false;
        }
        request.setAttribute("idempotentKey", idempotentKey);
    }
}

```

```

        return true;
    }
    @Override
    public void afterCompletion(HttpServletRequest request, HttpServletResponse response,
                               Object handler, Exception ex) {
        String idempotentKey = (String) request.getAttribute("idempotentKey");
        if (idempotentKey != null) {
            if (ex == null) {
                idempotentService.unlock(idempotentKey);
            }
        }
    }
    private String generateKey(HttpServletRequest request, Idempotent idempotent,
                               HandlerMethod handlerMethod) {
        Map<String, Object> variables = new HashMap<>();
        variables.put("request", request);
        variables.put("method", handlerMethod.getMethod().getName());
        variables.put("uri", request.getRequestURI());
        variables.put("ip", getClientIp(request));
        request.getParameterMap().forEach((k, v) ->
            variables.put(k, v.length > 0 ? v[0] : null));
        return idempotentService.generateKey(idempotent.key(), variables);
    }
    private String getClientIp(HttpServletRequest request) {
        String ip = request.getHeader("X-Forwarded-For");
        if (ip == null || ip.isEmpty() || "unknown".equalsIgnoreCase(ip)) {
            ip = request.getHeader("Proxy-Client-IP");
        }
        if (ip == null || ip.isEmpty() || "unknown".equalsIgnoreCase(ip)) {
            ip = request.getHeader("WL-Proxy-Client-IP");
        }
        if (ip == null || ip.isEmpty() || "unknown".equalsIgnoreCase(ip)) {
            ip = request.getRemoteAddr();
        }
        return ip;
    }

```

```

    }
    private void writeResponse(HttpServletResponse response, ErrorCode errorCode, String message) {
        try {
            response.setContentType("application/json;charset=UTF-8");
            response.setStatus(errorCode.getHttpStatusCode());
            ApiResponse<?> apiResponse = ApiResponse.error(errorCode, message);
            response.getWriter().write(
                new com.fasterxml.jackson.databind.ObjectMapper()
                    .writeValueAsString(apiResponse)
            );
        } catch (Exception e) {
            log.error("â\206\231â\205¥â\202ç-\211â\223\215â°\224â±è'¥", e);
        }
    }
}

package com.yiyundao.compensation.entity;
import com.baomidou.mybatisplus.annotation.*;
import lombok.Data;
import java.time.LocalDateTime;
@Data
@TableName("scheduled_task")
public class ScheduledTask {
    @TableId(type = IdType.AUTO)
    private Long id;
    @TableField("task_key")
    private String taskKey;
    @TableField("task_name")
    private String taskName;
    @TableField("task_group")
    private String taskGroup;
    @TableField("cron_expression")
    private String cronExpression;
    @TableField("description")
    private String description;
    @TableField("status")

```

```

private TaskStatus status;
@TableField("retry_count")
private Integer retryCount;
@TableField("max_retry_count")
private Integer maxRetryCount;
@TableField("retry_interval_seconds")
private Integer retryIntervalSeconds;
@TableField("last_execute_time")
private LocalDateTime lastExecuteTime;
@TableField("next_execute_time")
private LocalDateTime nextExecuteTime;
@TableField("last_result")
private String lastResult;
@TableField("alarm_enabled")
private Boolean alarmEnabled;
@TableField("alarm_receivers")
private String alarmReceivers;
@TableField("handler_bean")
private String handlerBean;
@TableField(value = "create_time", fill = FieldFill.INSERT)
private LocalDateTime createTime;
@TableField(value = "update_time", fill = FieldFill.INSERT_UPDATE)
private LocalDateTime updateTime;
@TableLogic
@TableField("deleted")
private Integer deleted;
public enum TaskStatus {
    PAUSED,
    RUNNING,
    FAILED,
    SUCCESS
}
}
package com.yiyundao.compensation.entity;
import com.baomidou.mybatisplus.annotation.*;

```



```

import lombok.Data;
import lombok.EqualsAndHashCode;
import java.time.LocalDateTime;
@Data
@EqualsAndHashCode(callSuper = false)
public abstract class BaseEntity {
    @TableId(type = IdType.AUTO)
    private Long id;
    @TableField(fill = FieldFill.INSERT)
    private LocalDateTime createTime;
    @TableField(fill = FieldFill.INSERT_UPDATE)
    private LocalDateTime updateTime;
    @TableField(fill = FieldFill.INSERT)
    private String createBy;
    @TableField(fill = FieldFill.INSERT_UPDATE)
    private String updateBy;
    @TableLogic
    @TableField(fill = FieldFill.INSERT)
    private Integer deleted;
    @Version
    private Integer version;
}
package com.yiyundao.compensation.entity;
import com.baomidou.mybatisplus.annotation.*;
import lombok.Data;
import java.time.LocalDateTime;
@Data
@Table("scheduled_task_execution")
public class ScheduledTaskExecution {
    @TableId(type = IdType.AUTO)
    private Long id;
    @TableField("task_id")
    private Long taskId;
    @TableField("task_key")
    private String taskKey;
    @TableField("start_time")

```

```
private LocalDateTime startTime;
@TableField("end_time")
private LocalDateTime endTime;
@TableField("duration_ms")
private Long durationMs;
@TableField("status")
private ExecutionStatus status;
@TableField("result")
private String result;
@TableField("error_message")
private String errorMessage;
@TableField("trace_id")
private String traceId;
@TableField(value = "create_time", fill = FieldFill.INSERT)
private LocalDateTime createTime;
public enum ExecutionStatus {
    RUNNING,
    SUCCESS,
    FAILED,
    RETRYING
}
}
```